

Generating Context-Aware Contrastive Explanations in Rule-based Systems

Lars Herbold
University of Cologne
Berlin, Germany
lars.herbold@outlook.de

Mersedeh Sadeghi
University of Cologne
Cologne, Germany
sadeghi@cs.uni-koeln.de

Andreas Vogelsang
University of Cologne
Cologne, Germany
vogelsang@cs.uni-koeln.de

ABSTRACT

Human explanations are often contrastive, meaning that they do not answer the indeterminate “Why?” question, but instead “Why P, rather than Q?”. Automatically generating contrastive explanations is challenging because the contrastive event (Q) represents the expectation of a user in contrast to what happened. We present an approach that predicts a potential contrastive event in situations where a user asks for an explanation in the context of rule-based systems. Our approach analyzes a situation that needs to be explained and then selects the most likely rule a user may have expected instead of what the user has observed. This contrastive event is then used to create a contrastive explanation that is presented to the user. We have implemented the approach as a plugin for a home automation system and demonstrate its feasibility in four test scenarios.

CCS CONCEPTS

- **Software and its engineering** → **Extra-functional properties;**
- **Human-centered computing** → *Interactive systems and tools; Ubiquitous and mobile computing.*

KEYWORDS

Explainability, Software Engineering, Smart Environments

ACM Reference Format:

Lars Herbold, Mersedeh Sadeghi, and Andreas Vogelsang. 2024. Generating Context-Aware Contrastive Explanations in Rule-based Systems. In *2024 Workshop on Explainability Engineering (ExEn '24)*, April 20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3648505.3648507>

1 INTRODUCTION

As smart cyber-physical systems, complex decision-making, and autonomous systems have advanced rapidly in recent years, the need for explainability in these domains has also increased. Explainability is a crucial research area that aims to provide transparent and understandable reasons for the outcomes and behaviors of these systems [6]. Users need to understand how the system works

and why it behaves in certain ways, otherwise, they may lose confidence and trust in it. This can result in negative consequences such as misuse or rejection of the system [18]. One way to address this issue is to *explain* the system’s rationale and outcomes to the users. This approach has been proven to work well in previous studies: A common finding in the literature is that trust and interaction quality between humans and computers can be improved by providing explanations [30].

Contrastive explanations clarify why an event occurred in contrast to another. They are intuitive for humans to produce and comprehend [15, 19]. There is evidence that explanations by humans are often contrastive [3, 21]. Contrastive explanations can provide a more accurate and fine-grained interpretability of system decisions. User-friendly explanations are important in smart environments because they can help users to better understand and control the technology, which can drive the adoption of smart technology [8, 25]. Numerous studies support the view that an explanation is not a one-size-fits-all concept. Instead, an individual’s characteristics, expectations, prior knowledge, and other contextual factors play a role in determining what they consider to be a satisfactory explanation [11, 17, 20]. Smart environments can enhance user engagement and satisfaction, and ultimately achieve greater success, by offering explanations that are tailored to the individual and relevant to the context [11].

Despite the theoretical and empirical studies that show the value of contrastive explanations [20, 21, 25, 30], there is a lack of approaches that show how to generate contrastive explanations for users. Some work has been done in the context of Explainable AI (XAI) [7, 29]. Explanation generation approaches for rule-based smart environments have, so far, focused on generating causal explanations (i.e., explaining the reasons for a happened event) [13, 22]. This paper proposes an approach for generating context-aware contrastive explanations in smart environments. Users generally need explanations after an event has occurred that does not align with their expectations [19]. Therefore, conservative explanations are inherently context- and user-specific. Our approach analyzes the situation in which an explanation is requested and predicts a possible expected event by a user. This contrastive event is then used to create a contrastive explanation that is presented to the user. We have implemented the approach as a plugin for a home automation system and demonstrate its feasibility in four test scenarios.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ExEn '24, April 20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0596-0/24/04

<https://doi.org/10.1145/3648505.3648507>

2 BACKGROUND

2.1 Explainability and its Engineering

Explainability (i.e., the ability of a system to provide explanations for its behavior) has been framed as a key requirement for software-supported decisions and a means of promoting transparency [6]. Explanations are seen as an option to mitigate the lack of transparency in a system. They can improve the understanding of a system by conveying information, thus influencing interpretability or understandability. Explainability engineering covers all activities related to making a system explainable [5]. This includes the elicitation and specification of explanation needs and related requirements [28?], the (automatic) detection of explanation needs at runtime, the construction of context-aware explanations, and a user-specific presentation.

2.2 Contrastive Explanations

Contrastive explanations are one type of explanation that explains why an event P occurred rather than event Q [16]. In common models of contrastive explanations, such as those of Lipton [19] and Miller [21], there are two central concepts:

- A *fact* is a piece of information that is considered to be true and can be verified. In the context of scientific explanations, a fact refers to a specific aspect of an event or phenomenon that is being explained. It is not simply the event or phenomenon itself, but a particular characteristic or feature of it that is of interest.
- A *foil* is an alternative scenario or situation that is used as a point of comparison to the fact. In the contrastive analysis of an explanation, the foil is used to focus the explanation by providing a further restriction on explanatory causes. The same fact may have several different foils, and the causes that explain a fact relative to one foil will not generally explain it relative to another. For example, when explaining why leaves turn yellow in November, we may not be interested in explaining this phenomenon in general, but only in comparison to a specific alternative scenario, such as why they turn yellow in November rather than in January or why they turn yellow rather than turning blue.

The *fact* relates to a situation *as is*. The *foil* to an imaginary situation, that could have been. Lewis states that a contrastive explanation provides information for why the *fact* occurred, but the *foil* did not [16]. Related to contrastive explanations are counterfactual explanations, which explain which past events would have caused the *foil* to materialize [26].

2.3 Smart Environments and Rule-based Systems

A *smart environment* is a physical space that has been enhanced with technology, such as sensors, actuators, and computing devices, to gather and use information about the environment and its occupants to adapt to their needs and preferences [10]. Alam et al. [2] describe a smart environment's abilities to include assisting inhabitants to live independently and comfortably with the help of technology and providing context-aware automated or assistive services for the user. This usually includes automating tasks with the help of this technology [10, 22]. In our work, we refer to smart homes and smart offices as smart environments although the term

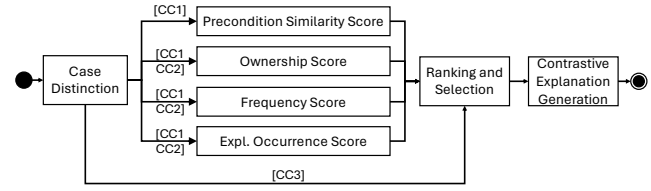


Figure 1: Overview of our approach for contrastive explanation generation. Depending on three possible confusing cases (CC), different scores are ranked to select the most likely expected alternative event.

is used more broadly in other studies and can include applications such as smart farming, smart cities, smart education, or smart factories. Smart environments can incorporate a set of *rules* to automate and control various aspects of its physical space. Similar to the concept of Nandi and Ernst [22], a rule consists of two sets: *Preconditions* and *actions*. A *precondition* is a binary indicator about the state of a part of the smart environment. Preconditions can be combined by logical operators to form more complex preconditions. When the precondition of a rule is fulfilled, the rule *fires* and executes an *action*. Systems that rely on a set of rules to execute their decisions are called *rule-based systems*.

3 APPROACH

The main challenge for building a contrastive explanation is identifying the foil, which represents the user's expectation in contrast to what happened (the fact). The foil is usually user- and context-specific, i.e., for the same fact, the foil may differ depending on the user and the situation. In rule-based systems, facts and foils refer to actions that the system performed as a result of a rule being fired (or not fired in the case of a foil). Therefore, in our approach, a foil always refers to an action that was not performed by the system but that was expected by the user. The objective of our approach is to identify the action and the corresponding rule of a rule-based system that represents the foil for a user in a specific situation. The identification of the need for a (contrastive) explanation is not in the scope of our approach. That means we assume that our approach is triggered in appropriate situations (e.g., by a user requesting an explanation or the system detecting a need for an explanation [4]). The identification of the fact is also not in the scope of our approach. We assume that the identification of the fact (the action that caused the explanation need) is straightforward as it may be the last action that happened or a specific action for which the user requests an explanation. In earlier work, we have provided a conceptual model to determine the *fact* in smart environments [27].

Figure 1 provides an overview of our approach. In the first step, we distinguish between three different confusing cases (CC) in which a contrastive explanation may be requested. These three different cases determine the set of factors that may influence the determination of the foil. Our approach then considers four factors that assign scores to all existing rules in the system. Each score contributes to the overall likelihood that a specific rule of the system was expected by a user in a certain situation. In the *ranking and selection* step, the approach accumulated the scores and selects the

most likely expected rule. Based on this rule, of which the action represents the foil, an explanation can be generated that contrasts the fact (what happened) with the prospective foil (what was expected instead). In the following, we describe the steps in detail.

3.1 Case Distinction based on the *Fact*

We identified three basic *confusing cases* (*CC*) where the user may expect a different outcome. In each case, the user expects an event to happen as a result of a rule being fired. The *fact*, in turn, is different in the three confusing cases.

CC1: *An action occurred but another action was expected:* The user is confused by an action of a fired rule that is different from the user's expectations. In this case, the *fact* is the action of the rule that was fired, and the *foil* is the expected action.

CC2: *No action occurred but an action was expected:* The user is confused because the system did not perform an expected action. This may be due to two reasons. Either the preconditions of the expected rule were not met and the rule was not fired (and no other rule regarding the device), or the *expected rule* does not exist anymore in the system (e.g., it was deleted). We will not go into detail about the case if a rule was deleted but outline later (Section 4.3) what could be done in future work. In both cases, the *fact* is "nothing," and the *foil* is the expected action.

CC3: *An error occurred but an action was expected:* An error occurred, so the resulting situation is different from the user's expectation. In this case, the *fact* is the occurring error, and the *foil* is the expected action.

In each case, a phenomenon occurred that caused the underlying confusing situation. In general, we call this the *happened event* and treat it as an equivalent to the *fact* in our smart environment setting.

3.2 Calculating Expected Rule Scores

To determine the *expected rule*, we define four factors that may influence the user's expectation:

- (1) **Precondition Similarity:** Actions of rules with similar preconditions may be more likely to be expected.
- (2) **Ownership:** Actions of rules that have been created by a user may be more likely to be expected.
- (3) **Frequency:** Actions of rules that fire frequently may be more likely to be expected.
- (4) **Explanation Occurrence:** Actions of rules that have been explained before may be more likely to be expected.

In *CC1*, all four factors can be applied to assess the *candidate rules*. Since *precondition similarity* can only be calculated if the *fact* is a rule, it cannot be used in the other cases. For *CC2*, the remaining three factors are applied. In case of an error (*CC3*), we know that the confusion was caused by the appearance of the error. Thus, we determine the *expected rule* as the last rule fired before the error occurred. Therefore, we do not consider any *expected rule* score in this case. The following paragraphs provide a detailed explanation of each factor and what value it contributes to a candidate rule.

Precondition Similarity. Actions that result from rules with similar preconditions to the rule that caused the observed action (the *fact*) are good candidates for a foil because a user may falsely

assume that the preconditions of another rule are met and therefore, they expect the action of that other rule. Precondition similarity can only be applied in situation *CC1*, where the *fact* is the result of a rule that was fired. In our approach, we compare all rules of a system with the rule that created the fact and calculate a precondition similarity score. The more similar the preconditions of two different rules are, the more likely it is that the user expects one of these rules if the other is fired.

To determine the similarity between the precondition of the candidate rule and the happened rule $s(CR, HR)$, we examine the set of unique conditions in the precondition of the candidate rule P_{CR} and the set of unique conditions in the precondition of the happened rule P_{HR} . We calculate the Jaccard similarity between these two sets, which is defined as:

$$Jaccard(P_{CR}, P_{HR}) = \frac{|P_{CR} \cap P_{HR}|}{|P_{CR} \cup P_{HR}|}$$

Consider an example with two rules: Rule *A* (the candidate rule) turns a fan on if the temperature in the room is higher than 25°C and someone is in the room. Rule *B* (the happened rule) turns the air conditioning on if the temperature in the room is higher than 25°C, it is a summer month, and someone is in the room. Rule *A* has two preconditions ($|P_A| = 2$) and rule *B* has three preconditions ($|P_B| = 3$). The cardinality of the union of preconditions of both rules is 3 ($|P_A \cup P_B| = 3$). Of these three preconditions, rule *A* and rule *B* share two ($|P_A \cap P_B| = 2$). The precondition similarity of *A* and *B* is now calculated as follows:

$$s(A, B) = Jaccard(A, B) = \frac{|P_A \cap P_B|}{|P_A \cup P_B|} = \frac{2}{3}$$

Ownership. If the *explainee* created a rule in the system, it is more likely that they expect this rule to be fired. We call the creating user of a rule the *owner* of the rule. Because of the ownership, we assume, they have a good understanding of the rule's functionality. Ownership is a binary measure with a value of 1 if the user is the owner of the rule or 0 if the user is not the owner of the rule.

Frequency. The more often a rule is fired by a system, the more likely it is that the explainee has experienced the rule's actions before. A higher frequency therefore makes a rule more expected by users than rules that are rarely fired and therefore have not been experienced as often by users. Based on the history of fired rules, we assign the number of times a rule was fired in a certain period as the frequency score.

Explanation Occurrence. If the system has already explained a rule to the user before, the user is more aware of the existence and functionality of this rule. Thus, the user is more likely to expect this rule in comparison to other, less explained rules. Like the frequency score, we assign the number of times a rule has been explained in a certain period as the occurrence score.

3.3 Deciding for an Expected Rule

Each rule of the system is now characterized by a set of values that indicate the expectation likelihood based on the set of up to four factors mentioned above. These values now need to be weighed and compared to each other to derive a ranked list of rules and finally decide on the most likely expected rule. This is a typical multi-criteria decision-making problem and we apply TOPSIS to rank the

alternatives [14, 31]. TOPSIS is based on the concept that the chosen alternative should have the shortest geometric distance from the positive ideal solution and the longest geometric distance from the negative ideal solution. TOPSIS compares a set of alternatives by normalizing scores for each criterion and calculating the geometric distance between each alternative and the ideal alternative, which is the best score in each criterion. The following steps of TOPSIS are required to decide which of the candidate rules is the *most likely rule*.

- (1) Create an evaluation matrix consisting of m alternatives (the rules) and n criteria (the influencing factors), with the intersection of each alternative and criteria given as x_{ij} , we therefore have a matrix $(x_{ij})_{m \times n}$.
- (2) The matrix values $(x_{ij})_{m \times n}$ are normalized for each column using the following normalization method

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{k=1}^m x_{kj}^2}}, i = 1, 2, \dots, m, j = 1, 2, \dots, n$$

resulting in the new normalized matrix $R = (r_{ij})_{m \times n}$.

- (3) For each column j , we determine the value of the best A_b and worst alternative A_w :

$$A_b = \{\max(t_{ij} | i = 1, 2, \dots, m)\} \equiv \{t_{bj} | j = 1, 2, \dots, n\}$$

$$A_w = \{\min(t_{ij} | i = 1, 2, \dots, m)\} \equiv \{t_{wj} | j = 1, 2, \dots, n\}$$

- (4) Calculate the L^2 -distance between the target alternative i and the worst condition A_w

$$d_{ib} = \sqrt{\sum_{j=1}^n (t_{ij} - t_{bj})^2}, i = 1, 2, \dots, m$$

and the distance between the alternative i and the best condition A_b

$$d_{iw} = \sqrt{\sum_{j=1}^n (t_{ij} - t_{wj})^2}, i = 1, 2, \dots, m$$

where d_{iw} and d_{ib} are L^2 -norm distances from the target alternative i to the worst and best conditions, respectively.

- (5) Calculate the similarity to the worst condition:

$$s_{iw} = \frac{d_{iw}}{d_{iw} + d_{ib}}, 0 \leq s_{iw} \leq 1, i = 1, 2, \dots, m$$

If alternative i is the worst alternative, $s_{iw} = 0$, if it is the best alternative, $s_{iw} = 1$.

- (6) The alternative with the highest s_{iw} is the *most likely rule* and therefore the one which we will choose to be the *expected rule*.

3.4 Explanation Generation

After the above-mentioned steps, we have determined the fact with its corresponding rule and the foil with its corresponding rule. To construct a contrastive explanation, we use a set of patterns that embed the fact and the foil into a comprehensive explanation. We use a different pattern for each confusing case (CC) (see Table 1).

- If the *happened event* is a rule (CC1), we follow a pattern that explains the contrast between the actions of the *happend rule* (A_{HR}) and the actions of the *expected rule* (A_{ER}). It also provides a reason why the *happened rule* was fired as it is aware of the preconditions of the *happened rule* (P_{HR}).

Table 1: Explanation Patterns by Confusing Case

Case	Pattern
CC1	$\langle A_{HR} \rangle$ occurred instead of $\langle A_{ER} \rangle$ because $\langle P_{HR} \rangle$
CC2	$\langle A_{ER} \rangle$ did not occur because $\neg P_{ER}$
CC3	$\langle A_{ER} \rangle$ did not occur because error $\langle HE \rangle$ occurred.

- If no event happened (CC2), we follow a pattern, which states that the actions of the *expected rule* (A_{ER}) did not occur because the preconditions of this rule (P_{ER}) were not met.
- If the *happened event* is an error (CC3), we follow a pattern that states that the expected actions (A_{ER}) did not happen because of the error (HE).

To streamline the grammatical structure of the final explanation, we provide an LLM with the pattern and the respective constituents and prompt it to generate a grammatically correct explanation without changing its meaning.

4 IMPLEMENTATION AND FIRST EVALUATION

We have implemented the approach as a plugin¹ for our *SmartEx* explanation engine. *SmartEx* is an extensible framework for adding explainability to smart environment systems. It integrates with Home Assistant², a popular rule-based engine for home automation, and provides explanation detection and generation capabilities. Our prototype extends *SmartEx* by adding explanations with a contrastive element.

To test the feasibility of our approach, we have created a set of test cases for our prototype. This prototype has undergone extensive testing in our *smart environments lab*. The lab uses Home Assistant to automate tasks in its smart environment. It contains a context manager, a service outside of Home Assistant and *SmartEx*, that provides contextual information about users.

We utilize two methods of how users can request explanations in the *smart explanation lab*: Scanning an NFC tag attached to a device or filling out a form for explanation requests available as a widget on one of Home Assistant's dashboards. With both methods, the user gives the device they are requesting the explanation for as input at the beginning of the generation process. When this generation is completed, the explanation can be presented to users in several forms: It can be read out loud by the lab's voice controller and can be displayed on the dashboard of Home Assistant on mobile applications, or in web browsers.

4.1 Evaluation Scenario

The underlying scenario for our test cases is a typical office situation. At the door to the office meeting room, an LED status light is installed, which is connected to the smart office management system. Several rules have actions that change the state of the status light. Table 2 provides an overview of the rules that are important for the scenario. Alice and Bob are coworkers in the office and can set rules for the smart office manager. Additionally, to demonstrate

¹<https://github.com/ExmartLab/SmartEx-Engine/tree/contrastive>

²<https://www.home-assistant.io/>

Table 2: Rules of the test scenario

Rule	Description	Owner
Meeting room not occupied	If the meeting room is not occupied, the status light is turned green	Alice
Meeting room occupied	If the meeting room is occupied, the status light is turned orange	Alice
Rain at lunch	If rain is expected at lunchtime, all status lights are turned blue	Bob
Sun at lunch	If sunny weather is expected at lunchtime, all status lights are turned orange	Bob
Danger	If a dangerous situation arises, all status lights are turned orange	Alice
Closing time	In the evening, all lights are turned off	Alice

Table 3: Test cases

Test	Explainee	Fact	Foil
1	Alice	Orange light (meeting room occupied)	Green light (meeting room not occupied)
2	Bob	Meeting Room occupied	Rain at lunch
3	Bob	Error	Meeting Room occupied
4	Alice	Nothing	Meeting Room not occupied

a scenario with *CC3*, an *incorrect installation error* is also part of the test cases.

4.2 Test Cases

The test cases listed in Table 3 show real-world examples of how our prototype generates contrastive explanations. They highlight the impact of the context on the determination of the expected rule and the generation of the explanation. The test cases show that the prototype delivers context-aware contrastive explanations for a variety of scenarios. Each test case highlights a different aspect that the framework considers when generating the explanation.

Test Case 1. In test case 1, a meeting is held in the meeting room. Most of the people leave the room but a few stay in the room. The *Meeting Room occupied* rule is still in effect and keeps the status light orange. Alice sees the group of people in the hallway after exiting the meeting room. She takes this as a sign that the meeting is finished and wonders why the status light is still orange. According to Table 3, we assume that Alice expects the light to be green because she expected the rule *Meeting Room not occupied* to fire. After she asks *SmartEx* for an explanation, the system first determines the *fact* and finds that the rule *Meeting Room occupied* is the most recent event that happened regarding the status light. As the *happened event* is a rule (*CC1*), the system uses all four expectation factors to determine the *expected rule*. The rules *meeting*

Table 4: Expected rule scores for test case 1

Rule	Prec. Sim.	Own.	Freq.	Occ.
Meeting room not occupied	0.333	1	65	3
Rain at lunch	0	0	4	4
Closing time	0	1	90	0

Table 5: TOPSIS results for test case 1

Rule	d_{ib}	d_{iw}	s_{iw}
Meeting room not occupied	0.075	0.368	0.830
Rain at lunch	0.362	0.2	0.356
Closing time	0.320	0.262	0.450

room occupied, *sun at lunch* and *danger* are ruled out as candidates as they are either the *happened rule* or they have the same action as the *happened rule*. Each remaining rule is given a score from every expectation factor. Table 4 summarizes the scores of all expectation factors. The scores for frequency and explanation occurrence are just random assumptions.

TOPSIS evaluates this decision matrix with the steps outlined in Section 3.3. In Table 5, the distances of the alternatives to the best alternative (d_{ib}) and worst alternative (d_{iw}) and the final performance score s_{iw} are shown for the candidate rules (rounded to 3 decimal places). Because the performance score for the *Meeting room not occupied* rule is the highest, it is the *most likely rule* and the system therefore determines it to be the *expected rule*.

Thus, the system correctly determines that *Meeting room not occupied* is the *expected rule* as it is the one Alice expected originally. The system then generates the following contrastive explanation according to the pattern for *CC1*: *For the meeting room status light, an orange hue occurred instead of the expected green light because the occupancy of the meeting room was detected, and the contact sensor for the meeting room door is turned off.*

Test Case 2. Bob set a rule that turns all status lights in the office to blue if rain is expected at lunchtime to remind everyone to take a coat or umbrella when going out for lunch. Now it is lunchtime and he notices that it is raining outside. A meeting is currently going on in the meeting room and Alice's rule (*Meeting room occupied*) turned the meeting room status light orange. As Bob walks past the meeting room, he wonders and asks *SmartEx* for an explanation as he is confused by the orange status light. The system first determines the *fact* and finds that the rule *Meeting Room occupied* is the event that happened regarding the meeting room status light. As the *happened event* is a rule (*CC1*), the system uses all four expectation measures to determine the *most likely rule*. The rest of the computation is similar to test case 1. In the end, the *rain at lunch* rule is determined to be the expected rule and the the following explanation is generated: *For the meeting room status light, an orange hue occurred instead of the expected blue light because the occupancy of the meeting room was detected, and the contact sensor for the meeting room door is off.*

Test Case 3. After some maintenance work in the office, the status light was not installed properly again and therefore is not

connected correctly. Bob is late to a meeting in the meeting room. As he approaches the meeting room, he wonders why the light is off and not orange because he knows the meeting already started. He asks *SmartEx* for an explanation about the light. The system finds that the *happened event* is an *incorrect installation error*. As an error happened *CC3*, the system looks at the Home Assistant log and determines that the *expected rule* is the last rule that was fired with an action involving the status light. At the start of the meeting, the system fired the *Meeting room occupied* rule and tried to turn the status light orange but as it could not be reached because of the faulty connection, the error occurred. Knowing the *fact* and the *foil*, *SmartEx* generates the explanation based on the pattern for *CC3*: *The orange light did not occur because an incorrect installation error occurred in the meeting room status light*.

Test Case 4. One day, Alice comes into the office in the morning and wonders why the status light is off. She expects it to be green as she programmed a rule (meeting room not occupied) to show if the room is available. What she does not know is that the door was not fully closed so the contact sensor assumed the door was still open. She asks *SmartEx* for an explanation. The system finds that no rule has been fired that would have impacted the light (*CC2*), so it uses the expectation measures to determine the *expected rule*. Precondition similarity cannot be assessed as no rule was fired to compare the preconditions of the candidate rules.

Again, in this case, the system correctly determines that *Meeting room not occupied* is the *expected rule* as it is the one Alice expected originally. The system generates an explanation based on the pattern for *CC2*: *For the meeting room status light, the green light did not occur because there was motion in the meeting room or the contact sensor for the meeting room was not closed*.

4.3 Limitations

While we cover the entire process of the generation of the explanation, one area that is not addressed is the detection of an explanation need. The process of the framework starts with the existence of a need for an explanation. This can either be voiced by the explainee personally or be detected via a separate system.

The occasion that a rule does not exist anymore in the system is not covered by this framework. It would be a subcase of *CC2* as a non-existing rule cannot be fired and could be part of future work. The explainable system could have a database of past rules and merge those with the current rules into the set of candidate rules for the determination of the *expected rule* in *CC2*.

In general, TOPSIS also allows weighting criteria differently. For simplicity, we decided to weigh all criteria equally. For future work, the weighting schemes proposed and analyzed by [23] could assist in choosing different weights for the TOPSIS criteria.

Overall, this framework is a novel approach for the generation of context-aware contrastive explanations in smart environments. It delivers a comprehensive process from the determination of the *fact* and *foil* to the construction of the actual explanation as a sentence.

5 RELATED WORK

Currently, there only exist a few approaches for explaining the behavior of cyber-physical systems (CPS). Drechsler et al. [9] sketch the first steps toward a conceptual framework for self-explaining

CPS. They propose to add a layer for self-explanation, which includes an abstract model of the system. They also propose adjusting the granularity of explanations for different predefined user groups. The generated explanations are cause-effect chains for observable actions using the abstract model. Users can access these chains to understand the cause of actions.

Chiyah Garcia et al. [12] present a modified version of fault trees that can be used to extract explanations. They call this the *model of autonomy*, which captures possible states of a system and the potential reasons for this state. This model has to be developed manually by experts. In contrast, Plambeck et al. [24] propose to learn dependencies on external influences for CPS with decision trees. This approach automatically identifies relevant influences and extracts data-related explanations of the system behavior. Such decision trees were already proven to be valuable for generating *why* and *why-not* explanations [18].

Houzé et al. [13] propose a modular architecture for self-explainable smart homes. The architecture consists of local explanatory components (LECs) that monitor single components and explain their behavior in terms of component-independent abstractions – predicates and events. A generic central component, called *Spotlight*, coordinates the LECs to generate system-wide explanations.

Agrawal and Cleland-Huang [1] examine the User Interface (UI) design trade-offs associated with providing timely and detailed explanations of autonomous behavior for swarms of small Unmanned Aerial Systems (sUAS). They analyze the impact of UI design choices on human awareness of the situation. They provide actionable guidelines for effectively explaining the autonomous behavior of multiple sUAS.

6 CONCLUSIONS

We proposed an approach for generating contrastive explanations in rule-based systems. It is designed to generate an explanation based on the current situation of the user. The approach considers several contextual factors to determine the *foil*, the situation that the user expected instead of the *fact*. The approach identifies three basic confusing cases (*CC*) that can lead to a contrastive situation where the user expected a different outcome. The factors used by the approach include precondition similarity, ownership, frequency, and occurrence. The scores assigned to the candidate rules by these measures are being evaluated by a rating system to arrive at a final decision about the *most likely rule*. We used TOPSIS as a multi-criteria decision analysis method particularly suited for this type of problem. Overall, this framework provides a comprehensive and intelligent approach to generating context-aware contrastive explanations in smart environments. The approach does not address the detection of an explanation need, which can either be voiced by the explainee personally or detected via a separate system.

REFERENCES

- [1] Ankit Agrawal and Jane Cleland-Huang. 2021. Explaining Autonomous Decisions in Swarms of Human-on-the-Loop Small Unmanned Aerial Systems. *AAAI Conference on Human Computation and Crowdsourcing* 9 (2021), 15–26. <https://doi.org/10.1609/hcomp.v9i1.18936>
- [2] Muhammad Raisul Alam, Mamun Bin Ibne Reaz, and Mohd Alauddin Mohd Ali. 2012. A review of smart homes—Past, present, and future. *IEEE Transactions on Systems, Man, and Cybernetics, part C (applications and reviews)* 42, 6 (2012), 1190–1203. <https://doi.org/10.1109/tsmc.2012.2189204>

- [3] André Artelt, Fabian Hinder, Valerie Vaquet, Robert Feldhans, and Barbara Hammer. 2021. *Contrastive Explanations for Explaining Model Adaptations*. Springer International Publishing, 101--112. https://doi.org/10.1007/978-3-030-85030-2_9
- [4] Mathias Blumreiter, Joel Greenyer, Francisco Javier Chiyah Garcia, Verena Klos, Maïke Schwammberger, Christoph Sommer, Andreas Vogelsang, and Andreas Wortmann. 2019. Towards Self-Explainable Cyber-Physical Systems. In *ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE. <https://doi.org/10.1109/models-c.2019.00084>
- [5] Wasja Brunotte, Larissa Chazette, Verena Klös, and Timo Speith. 2022. *Quo Vadis, Explainability? – A Research Roadmap for Explainability Engineering*. Springer International Publishing, 26--32. https://doi.org/10.1007/978-3-030-98464-9_3
- [6] Larissa Chazette and Kurt Schneider. 2020. Explainability as a non-functional requirement: challenges and recommendations. *Requirements Engineering* 25, 4 (2020), 493--514. <https://doi.org/10.1007/s00766-020-00333-1>
- [7] Qianglong Chen, Feng Ji, Xiangji Zeng, Feng-Lin Li, Ji Zhang, Haiqing Chen, and Yin Zhang. 2021. KACE: Generating Knowledge Aware Contrastive Explanations for Natural Language Inference. In *59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.acl-long.196>
- [8] Jiaxin Dai, Chao Zhang, Dzmitry Aliakseyeu, Samantha Peeters, and Wijnand A Ijsselstein. 2023. The Effect of Explanation Design on User Perception of Smart Home Lighting Systems: A Mixed-method Investigation. In *CHI Conference on Human Factors in Computing Systems*. ACM. <https://doi.org/10.1145/3544548.3581263>
- [9] Rolf Drechsler, Christoph Luth, Goerschwin Fey, and Tim Guneyso. 2018. Towards Self-Explaining Digital Systems: A Design Methodology for the Next Generation. In *IEEE 3rd International Verification and Security Workshop (IVSW)*. IEEE. <https://doi.org/10.1109/ivsw.2018.8494900>
- [10] Doaa Mohey El-Din, Aboul Ella Hassanien, and Ehab E. Hassanien. 2020. *Smart Environments Concepts, Applications, and Challenges*. Springer International Publishing, 493--519. https://doi.org/10.1007/978-3-030-59338-4_24
- [11] Juliana J. Ferreira and Mateus S. Monteiro. 2020. *What Are People Doing About XAI User Experience? A Survey on AI Explainability Research and Practice*. Springer International Publishing, 56--73. https://doi.org/10.1007/978-3-030-49760-6_4
- [12] Francisco J. Chiyah Garcia, David A. Robb, X. Liu, Atanas Laskov, Patron Patron, and Helen Hastie. 2018. Explain Yourself: A Natural Language Interface for Scrutable Autonomous Robots. In *Explainable Robotic Systems Workshop (HRI)*. <https://doi.org/10.48550/arXiv.1803.02088>
- [13] Etienne Houzé, Ada Diaconescu, Jean-Louis Dessalles, and David Menga. 2022. A generic and modular reference architecture for self-explainable smart homes. In *IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. IEEE.
- [14] Ching-Lai Hwang, Young-Jou Lai, and Ting-Yun Liu. 1993. A new approach for multiple objective decision making. *Computers & operations research* 20, 8 (1993).
- [15] Alon Jacovi, Swabha Swayamdipta, Shauli Ravfogel, Yanai Elazar, Yejin Choi, and Yoav Goldberg. 2021. Contrastive explanations for model interpretability. *arXiv preprint arXiv:2103.01378* (2021).
- [16] David Lewis. 1986. Causal Explanation. In *Philosophical Papers Vol. II*, David Lewis (Ed.). Oxford University Press, 214--240.
- [17] Q. Vera Liao, Daniel Gruen, and Sarah Miller. 2020. Questioning the AI: informing design practices for explainable AI user experiences. In *CHI conference on human factors in computing systems*.
- [18] Brian Y Lim, Anind K Dey, and Daniel Avrahami. 2009. Why and why not explanations improve the intelligibility of context-aware intelligent systems. In *SIGCHI conference on human factors in computing systems*.
- [19] Peter Lipton. 1990. Contrastive explanation. *Royal Institute of Philosophy Supplements* 27 (1990).
- [20] Tim Miller. 2019. Explanation in artificial intelligence: Insights from the social sciences. *Artificial intelligence* 267 (2019).
- [21] Tim Miller. 2021. Contrastive explanation: A structural-model approach. *The Knowledge Engineering Review* 36 (2021).
- [22] Chandrakana Nandi and Michael D Ernst. 2016. Automatic trigger generation for rule-based smart homes. In *ACM Workshop on Programming Languages and Analysis for Security*.
- [23] David L Olson. 2004. Comparison of weights in TOPSIS models. *Mathematical and Computer Modelling* 40, 7 (2004).
- [24] Swantje Plambeck, Goerschwin Fey, Jakob Schyga, Johannes Hinckeldeyn, and Jochen Kreutzfeldt. 2022. Explaining Cyber-Physical Systems Using Decision Trees. In *2nd International Workshop on Computation-Aware Algorithmic Design for Cyber-Physical Systems (CAADCPs)*. IEEE. <https://doi.org/10.1109/caadcps56132.2022.00006>
- [25] Michaela R Reisinger, Sebastian Prost, Johann Schrammel, and Peter Fröhlich. 2022. User requirements for the design of smart homes: dimensions and goals. *Journal of Ambient Intelligence and Humanized Computing* (2022).
- [26] Neal J Roesse. 1997. Counterfactual thinking. *Psychological bulletin* 121, 1 (1997).
- [27] Mersedeh Sadeghi, Lars Herbold, Max Unterbusch, and Andreas Vogelsang. 2024. SmartEx: A Framework for Generating User-Centric Explanations in Smart Environments. In *IEEE International Conference on Pervasive Computing and Communications (PerCom)*. IEEE.
- [28] Mersedeh Sadeghi, Verena Klös, and Andreas Vogelsang. 2021. Cases for Explainable Software Systems: Characteristics and Examples. In *IEEE 29th International Requirements Engineering Conference Workshops (REW)*. <https://doi.org/10.1109/REW53955.2021.00033>
- [29] Ilia Stepin, Jose M Alonso, Alejandro Catala, and Martín Pereira-Fariña. 2021. A survey of contrastive and counterfactual explanation generation methods for explainable artificial intelligence. *IEEE Access* 9 (2021).
- [30] Michael Winikoff. 2018. Towards trusting autonomous systems. In *5th International Workshop on Engineering Multi-Agent Systems (EMAS 2017)*. Springer.
- [31] RM Zulqarnain, M Saeed, N Ahmad, F Dayan, and B Ahmad. 2020. Application of TOPSIS method for decision making. *IJSRMSS International Journal of Scientific Research in Mathematical and Statistical Sciences* (2020).